scientific reports

Check for updates

ABIERTO

Detección de vulnerabilidades en código fuente Java usando una red neuronal convolucional cuántica con agrupación autoatenta, secuencia profunda y extracción de características híbridas basadas en grafos

Shumaila Hussain 1,2*, Muhammad Nadeem 3, Junaid Baber 2,4, Mohammed Hamdi 5, Adel Rajab 5, Mana Saleh Al Reshan 6 y Asadullah Shaikh 6

Las vulnerabilidades de software suponen una amenaza significativa para la seguridad del sistema, lo que exige métodos de detección automática efectivos. Las técnicas actuales enfrentan desafíos como problemas de dependencias, sesgo de lenguaje y granularidad de detección gruesa. Este estudio presenta un nuevo sistema de detección de vulnerabilidades basado en aprendizaje profundo para código Java. Aprovechando la extracción de características híbrida mediante técnicas basadas en grafos y en secuencias mejora la comprensión semántica y sintáctica. El sistema utiliza grafos de flujo de control (CFG), árboles de sintaxis abstracta (AST), dependencias de programa (PD) y vectorización por coincidencia más larga voraz para la representación de grafos. Una red neuronal híbrida (GCN-RFEMLP) y el modelo CodeBERT preentrenado extraen características, alimentándolas a una red neuronal convolucional cuántica con agrupación autoatenta. El sistema aborda problemas como dependencia de información a largo plazo y granularidad de detección gruesa, empleando representación de código intermedio y código de segmento interprocedural. Para mitigar el sesgo del lenguaje, se emplea un conjunto de datos de referencia de aseguramiento de software de referencia. Las evaluaciones demuestran la superioridad del sistema, logrando una precisión del 99,2 % en la detección de vulnerabilidades, superando métodos de referencia. El enfoque propuesto aborda de forma integral las vulnerabilidades, incluyendo validación de entrada inapropiada, autorizaciones faltantes, desbordamiento de búfer, ataques de secuencias entre sitios (XSS) y inyecciones SQL listadas por enumeración de debilidades comunes (CWE).

Palabras clave Detección de vulnerabilidades, QCNN autoatenta, Extracción de características, GCN híbrida, Seguridad de software, CodeBERT

La pandemia de COVID-19 ha intensificado drásticamente el uso de aplicaciones informáticas, llevando a un aumento sin precedentes de vulnerabilidades de software. Según la base de datos nacional de vulnerabilidades (NVD), se reportaron 20,158 vulnerabilidades en 20211. Este crecimiento exponencial de las vulnerabilidades de seguridad está causando impactos económicos significativos y pérdidas financieras sustanciales2–7.

Por lo tanto, la detección de vulnerabilidades de software se ha vuelto más crucial y desafiante que nunca. La necesidad de enfoques de detección automática generalizados, escalables, precisos, de grano fino y de alta velocidad es evidente. Las vulnerabilidades suelen originarse por deslices de programación, que las herramientas de detección actuales, ya sea mediante análisis estático o dinámico del código, a menudo no abordan adecuadamente. El análisis de código para vulnerabilidades de seguridad sin ejecución es la técnica de análisis estático de código, mientras que en el análisis dinámico del código, se prueba la aplicación en ejecución para la vulnerabilidad de seguridad. Las técnicas de análisis estático pueden ser intensivas en recursos, mientras que dinámico

1 Departamento de Ciencias de la Computación, Universidad de Mujeres Sardar Bahadur Khan, Quetta, Pakistán. 2 Departamento de Ciencias de la Computación e Informática, Universidad de Balochistán, Quetta, Pakistán. 3 Higher Colleges of Technology, Abu Dhabi, Emiratos Árabes Unidos. 4 GIPSA-Lab, Universidad Grenoble Alpes, 38000 Grenoble, Francia. 5 Departamento de Ciencias de la Computación, Colegio de Ciencias de la Computación y Sistemas de Información, Najran University, 61441 Najran, Arabia Saudita. 6 Departamento de Sistemas de Información, Colegio de Ciencias de la Computación y Sistemas de Información, Najran University, 61441 Najran, Arabia Saudita. *correo: shumaila.hussain@sbkwu.edu.pk

el análisis de código puede aumentar el tiempo de ejecución y afectar negativamente al rendimiento8. Ambos enfoques son específicos del lenguaje, basados en reglas y dependientes del conocimiento de los desarrolladores, lo que los hace propensos a errores, sesgados, con grano grueso y llevando a tasas de falsos negativos inaceptablemente altas.

Las técnicas de aprendizaje automático (ML) han demostrado ser prometedoras en la evaluación de vulnerabilidades9–14. Las redes neuronales profundas (DNN) han demostrado capacidades para aprender patrones de código fuente, destacándose en la detección de errores a nivel de sintaxis y reconocimiento de patrones15–17. Sin embargo, las soluciones de aprendizaje profundo (DL) existentes para evaluaciones de vulnerabilidades tienen ciertas limitaciones; se centran principalmente en la estructura sintáctica del código, descuidando su información semántica18–20. Apuntan a un solo archivo de código fuente o a un pequeño conjunto de datos o se apoyan en APIs de interfaces de procesamiento de aplicaciones para abordar la vulnerabilidad seleccionada. Además, las técnicas de DL a menudo tienen dificultades para comprender las transferencias de valor dentro de los códigos fuente debido a la falta de información semántica, lo que resulta en una tasa alta de falsos positivos y un enfoque menos escalable21–23.

La red neuronal convolucional cuántica auto-atenta empleada junto con técnicas de aprendizaje profundo mejora significativamente el cuello de botella de memoria, la comprensión semántica de los patrones de código y acelera el rendimiento. El sistema propuesto de detección de vulnerabilidades puede detectar una amplia gama de vulnerabilidades, incluidas la validación de entrada inapropiada, ataques de inyección SQL, autorización faltante, secuencias de comandos entre sitios (XSS) y ataques de desbordamiento de búfer listados entre las 25 vulnerabilidades de seguridad más impactantes según la enumeración de debilidades comunes (CWE). La CWE es un proyecto de MITRE y es responsable de enumerar los tipos de debilidades de software y hardware según su impacto para ayudar a prevenir la vulnerabilidad. Este artículo de investigación contribuye al campo de la detección automática de vulnerabilidades de varias maneras significativas:

1. Desarrolla un sistema novedoso de detección de vulnerabilidades que implica una detección eficiente y precisa de vulnerabilidades mediante extracción de características híbridas al concatenar enfoques basados en grafos y basados en secuencias para hacer frente a patrones de vulnerabilidad complejos, mejorando la granularidad de la detección y reduciendo las tasas de falsos positivos.
2. Propone una red neuronal gráfica híbrida basada en GCN-RFEMLP para superar la ausencia de información de orden de los nodos en el grafo. Nuestro método envolvente fusionado ha reducido la dimensión de las características y eliminado características irrelevantes para mejorar la eficiencia.
3. Presenta un modelo CodeBERT preentrenado bimodal para implementar una extracción de características ajustada, reduciendo así la brecha semántica para mejorar la detección de vulnerabilidades.
4. Analiza el conjunto de datos de detección de vulnerabilidades y equilibra el conjunto de datos para evitar el sobreajuste, mejorando así el rendimiento.
5. Emplea el conjunto de referencia de aseguramiento de software SARD para entrenamiento y prueba del modelo, preprocesando los conjuntos de datos para lograr resultados optimizados. El sistema propuesto se prueba con cinco conjuntos de datos diferentes para garantizar su rendimiento, robustez y validez.
6. Emplea una novedosa red neuronal convolucional cuántica que utiliza agrupamiento autoatento para mejorar la computación, las dependencias a largo plazo y los problemas de cuello de botella de memoria para clasificar el código vulnerable y el tipo de vulnerabilidad. Según nuestro mejor conocimiento, el agrupamiento autoatento de QCNN se usa por primera vez para clasificar las vulnerabilidades.
7. Propone un marco novedoso para la selección efectiva de características, que contribuye a una comprensión más amplia de este campo y sugiere un enfoque más equilibrado y eficaz para la detección de vulnerabilidades en diversos tipos.

El resto de este artículo se estructura de la siguiente manera:
La sección de "Trabajos relacionados" profundiza en una revisión de la literatura relevante. La sección de "Metodología" describe la metodología empleada en esta investigación. La sección de "Experimentos y resultados" detalla el experimento y los resultados, incluyendo la configuración experimental y los resultados derivados del método propuesto. La sección de "Conclusión" ofrece las conclusiones extraídas de este estudio.

Trabajos relacionados

Auditoría manual de código fuente, que implica un equipo de expertos en seguridad, examina el código fuente en busca de vulnerabilidades, siendo el enfoque más tradicional para encontrar vulnerabilidades de software24. Sin embargo, las técnicas convencionales de análisis de vulnerabilidad de software a menudo tienen dificultades para lidiar con vulnerabilidades de seguridad de software en tiempo real y cada vez más presentes.

La detección de vulnerabilidades basada en análisis de código está en tendencia y se clasifica en tres enfoques principales: detección estática, híbrida y dinámica25. El análisis estático examina el código fuente sin ejecución, mientras que el dinámico lo analiza a través de la ejecución. El análisis híbrido combina ambos. Muchas herramientas y técnicas, como la comparación de código, ejecución simbólica y técnicas de inferencia, se han desarrollado para el análisis estático. Sin embargo, estas técnicas no cubren todas las vulnerabilidades existentes y están mal equipadas para analizar amenazas de seguridad emergentes. Las técnicas de análisis dinámico, que incluyen fuzzing y análisis de contaminación, requieren un tiempo computacional sustancial y recursos26–31. Además, el rendimiento y la fiabilidad de estos métodos no son suficientes para hacer frente a los desafíos de seguridad actuales.

El aumento de vulnerabilidades de software ha llevado a los investigadores a idear mejores estrategias de detección. Los investigadores de seguridad de software han comenzado a aprovechar el poder predictivo del aprendizaje automático para abordar estos desafíos de seguridad. Las técnicas de aprendizaje automático, ya sean supervisadas, no supervisadas o semi-supervisadas, se utilizan cada vez más para la detección de vulnerabilidades. Entre los diversos enfoques de aprendizaje automático, el aprendizaje supervisado es el ampliamente adoptado para la detección de vulnerabilidades de software. La Figura 1 ilustra los enfoques de aprendizaje supervisado para la evaluación de vulnerabilidades.

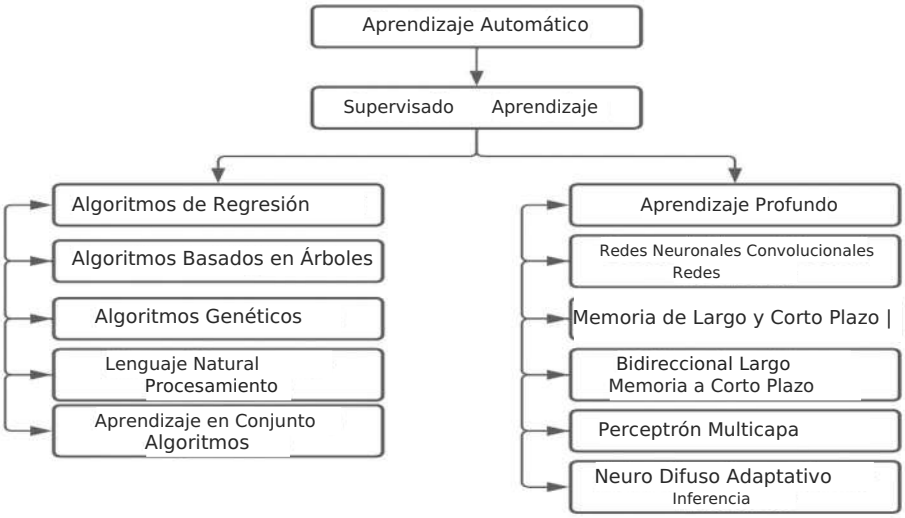

Figura 1. Técnicas de aprendizaje automático utilizadas para evaluaciones de vulnerabilidad.

Aprendizaje de representación de código

El código debe seguir un formato específico para implementar técnicas de aprendizaje automático, categorizadas en tres métodos principales de representación:

Basado en secuencias En este enfoque, los datos se dividen en fragmentos, como caracteres, tokens o API, utilizando técnicas como bag-of-words, n-gram, word2vec, etc. Estas técnicas implican preprocesamiento de datos, tokenización y adopción de redes neuronales. Sin embargo, pueden carecer de abstracción de contexto a largo plazo del código.

Basado en Árbol Este método emplea una estructura de red neuronal sobre una representación de datos basada en el árbol de sintaxis abstracta (AST). El árbol se subdivide en pequeñas declaraciones que contienen fragmentos de código. Los desafíos incluyen la complejidad de fragmentos de código y el desvanecimiento de gradientes.

Basado en Grafo Este enfoque representa el código en una estructura de grafo, principalmente utilizando un grafo de propiedades de código (CPG) compuesto por un árbol de sintaxis abstracta (AST), grafo de flujo de control (CFG), grafo de flujo de contexto (XFG) y grafo de dependencias de programa (PDG) para la representación de código intermedio32. Si bien las técnicas basadas en grafos pueden abordar dependencias a largo plazo, requieren una computación intensiva.

En una de las investigaciones relacionadas, un estudio de análisis de vulnerabilidad utilizó redes neuronales gráficas (GNN) y redes neuronales gráficas circulares para detectar el código vulnerable33,34. En otro estudio, los investigadores utilizaron un grafo de flujo para la representación del código fuente, realizaron vectorización mediante word2vec y aplicaron el método de red neuronal gráfica para identificar la vulnerabilidad35–37. El detector de vulnerabilidad de software llamado DeepVulSeeker utilizó un modelo preentrenado para convertir descripciones en lenguaje natural a código de programación. Otro estudio en contexto utilizó representación de código intermedio aplicando AST, CFG y DFG y desplegó un modelo preentrenado, mientras que redes neuronales CNN y FNN se utilizaron para clasificar la vulnerabilidad38,39. Las redes neuronales de árbol de sintaxis abstracta40,41 y la red neuronal profunda autocontenida con minería de texto también se probaron42. De manera similar, ChatGPT implica interacción humana para identificar vulnerabilidades y recomendar soluciones43.

Otro estudio exploró árboles de regresión para la detección de vulnerabilidades44. De manera similar, se utilizó un enfoque híbrido que emplea aprendizaje profundo para vulnerabilidades ligeras asistidas en un estudio relacionado, mientras que otra investigación utilizó aprendizaje de representación intermedia mínima45,46. Los investigadores explotan el corte de programas y la unidad recurrente con compuerta binaria (BGRU) en una naturaleza similar del estudio, mientras que el corte de código usando métricas de código como características se utiliza para detectar vulnerabilidades relacionadas con el uso de punteros47,48. Otros estudios implementaron técnicas de aprendizaje profundo como CNN y otras, junto con la selección de características, para detectar vulnerabilidades de SQL y cross-site scripting48–51. Otro estudio propuso un modelo basado en el aprendizaje de características de origen y clasificación52. Se ha observado que la selección de características se estudia con frecuencia junto con enfoques de aprendizaje automático para la detección de vulnerabilidades53–55.

Dos estudios similares utilizaron word2vec y LSTM para identificar código con cross-site scripting, inyección de SQL, falsificación intersitio y vulnerabilidad de redirección abierta56,57. El modelo de red neuronal recurrente llamado BiLSTM se utiliza para enfocarse en errores de búfer y detección de vulnerabilidades de gestión de recursos58. De manera similar, BiLSTM y taint analysis tuvieron un buen desempeño en una de las investigaciones llevadas a cabo en el mismo contexto59. Técnicas como CNN, memoria de corto y largo plazo (LSTM) y gráficos dirigidos se utilizaron para la detección de vulnerabilidades60.

Uno de los estudios relacionados comparó los métodos Random Forest, CNN y RNN para la detección de vulnerabilidad61. De manera similar, el modelo basado en GNN superó en detección de vulnerabilidades62,63. Otro estudio presentó un análisis comparativo utilizando Naïve Bayes, árboles de decisión, SVM, k-nearest neighbor y RF para evaluar el rendimiento de detección de vulnerabilidades de software64–67. Otro estudio centrado en SVM, clasificadores Naïve Bayes de multivariable, y codificadores bidireccionales basados en BERT para aprendizaje por transferencia concluyó que BERT superó a otros métodos en la detección de vulnerabilidades68. Cabe destacar que ninguno de los estudios revisados consideró la similitud semántica del código, destacando la brecha en las técnicas de aprendizaje profundo utilizadas para la detección de vulnerabilidades. En contraste, nuestro trabajo extrae la similitud semántica del código, mejorando el rendimiento del sistema, como se detalla más adelante en la sección de resultados.

Validación de entrada incorrecta, una de las principales causas de vulnerabilidad de seguridad en aplicaciones informáticas, puede activar ataques de inyección SQL, falta de autorización, ataques de cross-site scripting (XSS) y desbordamientos de búfer. La Enumeración de Debilidades Común (CWE) del proyecto Mitre, un diccionario integral de debilidades de software, clasificó la validación de entradas como la cuarta más frecuente y peligrosa vulnerabilidad de seguridad en 202169,70. Por lo tanto, seleccionamos la validación de entrada incorrecta, cross-site scripting, desbordamiento de búfer, falta de autorización y vulnerabilidad de inyección SQL, clasificadas entre las 25 vulnerabilidades de seguridad más impactantes y peligrosas listadas por CWE para evaluar nuestro sistema propuesto. La Tabla 1 a continuación muestra algunas técnicas de detección de vulnerabilidades comúnmente utilizadas para analizar la vulnerabilidad seleccionada.

Metodología

Esta sección describe nuestro sistema propuesto para la detección de vulnerabilidades, que introduce una extracción de características fusionadas que aprovecha la comprensión semántica y sintáctica del código para una evaluación de vulnerabilidades matizada.

Marco del sistema propuesto de detección de vulnerabilidades

La auditoría de código se realiza principalmente en lenguajes C/C+, aunque siempre hay espacio para auditoría de código Java debido a técnicas deficientes de auditoría de código cuantificadas para este lenguaje. Nuestro sistema busca detectar automáticamente vulnerabilidades de software a partir del código Java usando DL, considerando la estructura sintáctica y la semántica del código, enfocándose en la detección de vulnerabilidades de gran alcance. Dado que las técnicas DL existentes a menudo pasan por alto las relaciones semánticas en el código, nuestro sistema está diseñado para llenar ese vacío y mejorar la tasa de falsos positivos. El sistema propuesto utiliza un mecanismo novedoso basado en extracción de características híbrida que concatena extracción de características basada en secuencias y en grafos y detecta la vulnerabilidad mediante aprendizaje profundo.

La metodología propuesta se representa en la Fig. 2 a continuación. El esquema propuesto se divide en tres partes (1) Representación de entrada intermedia (2) Extracción de características híbrida (3) Clasificación. El primer paso implica un conjunto de datos estándar convertido en representación de código fuente utilizando grafos de propiedades de código y tokenización para que sea presentable y aprovechar las técnicas de aprendizaje automático. En el segundo paso, se aplica la extracción de características híbrida. La extracción de características de grafos utilizada junto con la extracción de características basada en secuencias aprovecha la estructura semántica y sintáctica del código. Las características extraídas se concatenan, y se emplea una red neuronal convolucional cuántica con agrupamiento autoatento para detectar vulnerabilidades seleccionadas.

Las vulnerabilidades seleccionadas se enumeran entre las más impactantes según CWE e incluyen validación de entrada incorrecta, vulnerabilidad de inyección SQL, falta de autorización, cross-site scripting y desbordamiento de búfer. El sistema detecta funciones vulnerables y tipos de vulnerabilidad.

Vulnerabilidad	Enfoque
Ataques XSS de cross-site scripting	1. Detección de ataques XSS por cross-site scripting usando un modelo CNN modificado71 2. Detección automatizada de ataques XSS del lado del servidor utilizando inyección de límites72 3. Análisis basado en rastreo de contaminación de DOM cross-site scripting denominado TT-XSS73 4. Se utiliza Máquina de Vector de Soporte para detectar vulnerabilidad de cross-site scripting ciega74 5. Reducción de superficies de ataque para ataques de cross-site scripting mediante SDLC seguro75 6. Detección de vulnerabilidad de cross-site scripting usando LSTM y redes neuronales recurrentes (RNN) llamadas D -eepXSS76 7. Uso de algoritmos genéticos y aprendizaje por refuerzo para la detección de ataques XSS77 8. Uso de ML con características híbridas para la detección de ataques XSS78 9. Uso de inferencia difusa para la detección dinámica de ataques XSS cross-site scripting79
Ataques de desbordamiento de búfer	1. Análisis de intrusiones de red para ataques de desbordamiento de búfer80 2. Implementación de función de biblioteca de cadenas para detectar desbordamiento de enteros a desbordamiento de búfer81 3. Detección estática de desbordamiento de búfer realizada y detección automática sugerida82 4. Detección estática de desbordamiento de búfer y reparación usando la herramienta BovInspector83
Ataques de inyección SQL	1. Detección de ataques de inyección SQL usando un árbol de decisión84 2. Uso de análisis de comportamiento y respuesta para ataques de inyección SQL85 3. Detección de ataques de inyección SQL en aplicaciones web mediante análisis heurístico86 4. Aplicación de técnicas neurodifusas para prevenir y detectar ataques de inyección SQL87 5. Algoritmo diseñado para pruebas de caja negra para mitigar vulnerabilidad de inyección SQL88 6. Una técnica basada en tráfico llamada DIAVA para detectar filtraciones de datos y ataques de inyección SQL89 7. Un método híbrido que consiste en aumentar tablas de bases de datos con símbolos, luego usar un algoritmo para consultas y otro algoritmo diseñado para la coincidencia de cadenas para prevenir y detectar la inyección SQL a -ataques90 8. Usar la aleatorización de conjuntos de intrusiones para detectar ataques de inyección SQL91 9. Se desarrolla una herramienta para detectar ataques de inyección SQL y mostrar sugerencias para solucionarlos92
Autorización faltante	1. La herramienta se desarrolla para detectar autorización faltante en sistemas en la nube distribuidos utilizando inferir vari-definición, datos propiedad del usuario y estado crítico del sistema93 2. La técnica propuesta basada en SE de role cast consiste en el contexto de eventos sensibles a la seguridad que son controlados por roles94 3. The Vanguard es un enfoque que consiste en análisis estático para operaciones sensibles, analizando el sostenimiento-habilidad mediante análisis de contaminación y la existencia de grado de riesgo de autorización faltante95 4. Se propone VRust para analizar vulnerabilidad, incluida la autorización faltante para Solana, asignando reglas de validación para cuentas de entrada vulnerables96 5. El sistema CRIX consiste en sistemas interprocedurales, semánticos y contextuales97 6. MACE se basa en comprobar la consistencia del estado de autorización98

Tabla 1. Técnicas comúnmente utilizadas para la detección de vulnerabilidades.

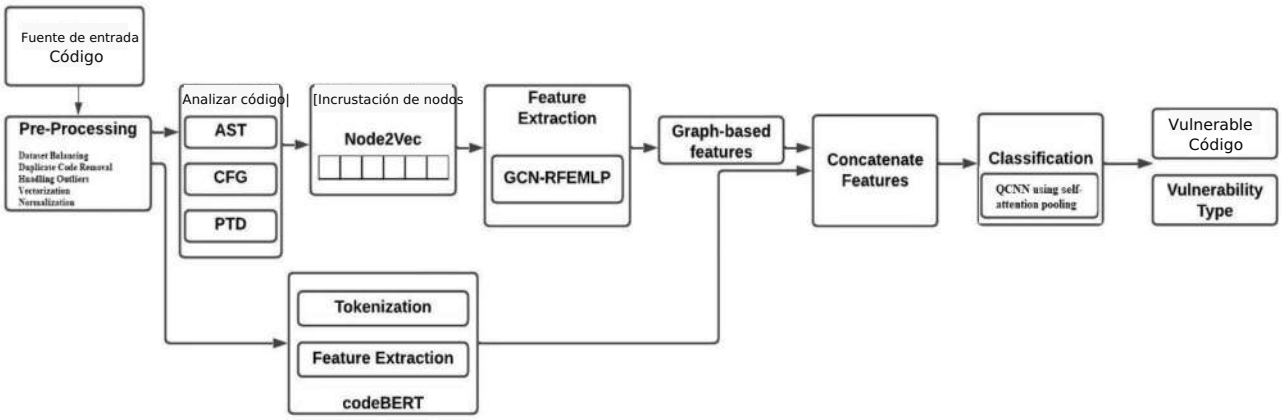

Figura 2. El marco del sistema propuesto de detección de vulnerabilidades.

Conjunto de datos/adquisición de datos

Para entrenar nuestro sistema propuesto, hemos utilizado el conjunto de datos de referencia Software Assurance Reference Dataset (SARD), que contiene cientos de miles de programas de código fuente con vulnerabilidades conocidas. Este conjunto de datos incluye 42,212 archivos que comprenden 29,258 muestras seguras y 12,954 muestras inseguras de código fuente, cubriendo 150 clases de errores o debilidades listadas por CWE99–104. Para nuestro estudio, hemos seleccionado 46,447 programas Java de SARD, incluidas vulnerabilidades relacionadas con ataques de inyección SQL, falta de autorización, cross-site scripting, validación de entrada incorrecta y desbordamiento de búfer. El sistema propuesto se valida utilizando otros conjuntos de datos de referencia, incluidos Juliet java 1.3105–107, FUNDED, Vul4j, CVEfixes y CodeXGLUE.

Preprocesamiento de conjuntos de datos

El preprocesamiento de datos implica varios pasos esenciales.

a. Equilibrio del conjunto de datos.

Abordar el desequilibrio del conjunto de datos es crucial para un rendimiento óptimo de los algoritmos de aprendizaje automático. El conjunto de datos de referencia para la detección de vulnerabilidades a menudo presenta una disparidad significativa entre códigos vulnerables y limpios. Lograr un conjunto de datos equilibrado es vital para un rendimiento preciso y eficiente del algoritmo, ayudando a reducir las tasas de falsos positivos. Además, los valores faltantes se manejan adecuadamente.

b. Eliminación de código duplicado

La eliminación de código duplicado mejora el rendimiento, reduce la complejidad y minimiza el tiempo de ejecución. Se utilizan árboles de decisión para la eliminación eficiente de código duplicado y clones de código.

c. Manejo de valores atípicos

La organización del conjunto de datos es fundamental para un mejor rendimiento. Los valores atípicos se detectan y se manejan de manera efectiva mediante transformación logarítmica, contribuyendo a la normalización del conjunto de datos.

d. Vectorización

Los datos textuales se transforman en forma numérica mediante vectorización, asegurando una escalación uniforme y mejorando el rendimiento del algoritmo.

e. Normalización

Una mayor normalización del conjunto de datos garantiza una escalabilidad consistente sin comprometer las diferencias de rango. La normalización de datos iguala el impacto de cada característica, abordando posibles problemas de precisión que surgen de valores grandes inherentes. Se emplea la técnica de Z-Scaling para la normalización de datos, convirtiendo conjuntos de datos basados en texto en enteros.

Extracción de características gráficas

Representación de código intermedio

Hemos aplicado el clásico grafo de propiedades de código (CPG) para la representación gráfica del código, que es una combinación de árbol de sintaxis abstracta (AST), grafo de flujo de control (CFG) y grafo de dependencia de programa. Ayuda a analizar la estructura sintáctica y la semántica del código. Es importante convertir el código en una representación de código intermedio para eliminar los puntos innecesarios y reducir las dependencias.

a. Árbol de sintaxis abstracta (AST)

El AST se utiliza para analizar eficazmente la estructura sintáctica del código. El árbol de sintaxis abstracta comprende un nodo raíz que contiene funciones, ramas de declaraciones, predicciones y expresiones, mientras que los nodos hoja representan los operadores, identificadores y palabras clave.

b. Grafo de flujo de control (CFG)

El CFG representa el orden de ejecución del código. Explica las sentencias y condiciones que deben cumplirse para la ejecución de las ramas de código. Los nodos en el CFG indican las sentencias, mientras que las aristas denotan la transferencia de control.

c. Grafo de dependencia de programa (PDG)

Describe las dependencias de control y de datos en la función. El borde de dependencia de datos mantiene la variable declarada para ser utilizada más tarde, mientras que los bordes de dependencia de control denotan el impacto de predicados en variables.

Incrustación de nodos

La incrustación de nodos tiene como objetivo reducir las propiedades de los nodos en vectores de menor dimensión. El resultado de la incrustación de nodos se alimenta como entrada a técnicas de procesamiento basadas en aprendizaje automático. La flexibilidad para explorar vecindades en node2vec se ha observado que proporciona una representación más rica. La rica información estructural mejora la capacidad de las características para implicar información no lineal. Por lo tanto, se usa node2vec para la incrustación de nodos con caminata aleatoria usando skip-gram con muestreo negativo para maximizar la probabilidad de preservar la vecindad de los nodos. node2vec es una cadena de Markov de segundo orden. Implementa caminatas aleatorias en grafos para extraer el par de contexto usando un enfoque de arranque y usarlos para entrenar el modelo word2vec. Transforma grafos en representación numérica manteniendo la estructura de la red de modo que los nodos cercanos permanezcan cercanos en la incrustación. La estructura de node2vec se muestra en la Fig. 3.

Extracción de características

Hemos empleado una red neuronal híbrida de grafos GCN-RFEMLP basada en una red neuronal convolucional de grafos (GCN) y un perceptrón multicapa fusionado con envoltorio de eliminación recursiva de características. La GCN carece de similitud de características, lo que puede crear ruido. Por lo tanto, hemos concatenado RFEMLP con GCN para superar este problema. La red neuronal convolucional de grafos está diseñada para tratar con datos de estructura de grafos. Implementa una técnica de paso de mensajes donde la información de incrustación de un nodo se actualiza en función del nodo vecino. La incrustación de nodos se convierte en incrustación de grafos, sirviendo como entrada a un clasificador completamente conectado. Hemos añadido una capa bi-afina en la GCN para lograr un mejor análisis de dependencias y preservar la semántica del código. La composición estructural de las redes neuronales convolucionales de grafos se ilustra en la Fig. 4 a continuación.

.

Usamos una red neuronal MLP con una función de activación lineal rectificada, ReLU, en la capa oculta y una función de activación Softmax en la capa de salida. La fórmula generalizada para ReLU se muestra en la Ecuación (1).

$$O = WA + B \tag{1}$$

donde O es la salida antes de aplicar la función de activación, W representa los pesos, A representa la entrada a la capa y B representa el sesgo.

La Fig. 5 ilustra laicomposición estructural de la red MLP. Hemos utilizado optimizadores adam, adadelta, momentum y descenso de gradiente estocástico (SDG) junto con funciones de pérdida de error cuadrático medio (MSE) y error absoluto medio (MAE) para seleccionar la mejor opción. Hemos emparejado cada optimizador con una función de pérdida para obtener los resultados.

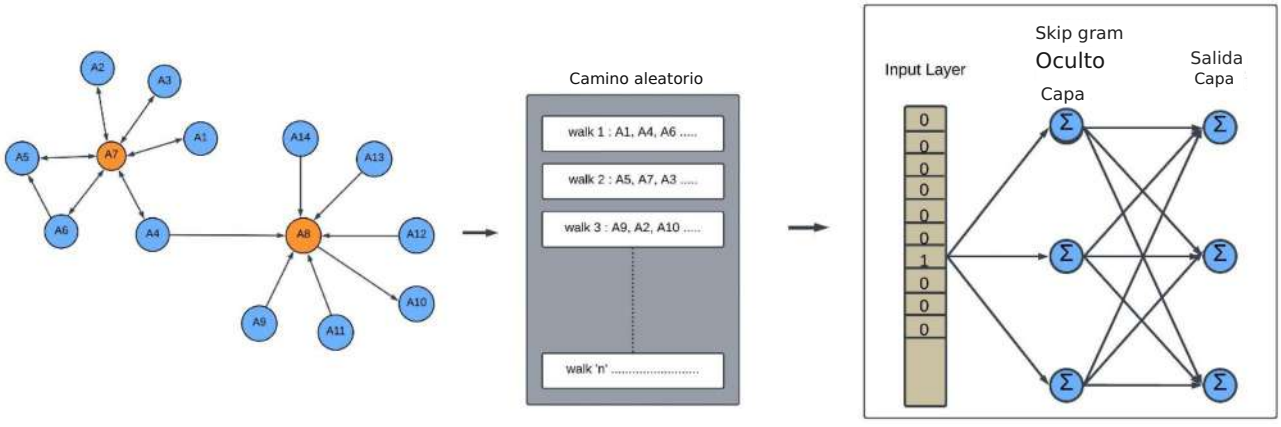

Figura 3. Estructura de Node2Vec usando caminata aleatoria y skip-gram.

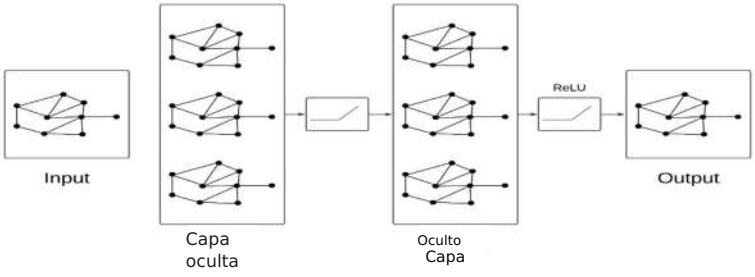

Figura 4. Estructura de la red neuronal convolucional de grafos.

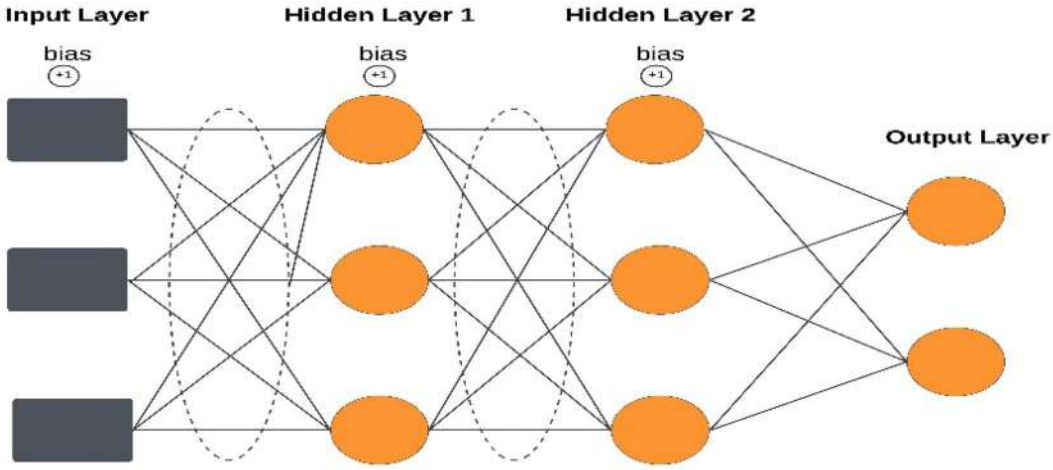

Figura 5. Estructura del modelo de red neuronal perceptrón multicapa (MLP).

Las selecciones dadas a continuación muestran la combinación de cada optimizador y función de pérdida. La Selección 1 muestra la combinación del optimizador Adam con la función de pérdida MSE de manera similar; La Selección 2 muestra la combinación del optimizador Adam con la función de pérdida MAE, y así sucesivamente.

La Tabla 2 representa diferentes composiciones de optimizadores y funciones de pérdida. Los resultados obtenidos de cada selec- ción se comparan para implementar la mejor combinación de optimizador y función de pérdida para mejorar la precisión del sistema. Hemos realizado experimentos para obtener la combinación óptima con la máxima reducción de pérdida para mejorar el rendimiento del algoritmo. La función de pérdida enumera la diferencia entre el valor real y el valor previsto. La selección 3 y la selección 7 mostraron resultados mejorados. Por lo tanto, hemos seleccionado la selección 7 para usar con MLP y mejorar el rendimiento. Además, el entrenamiento del modelo incluye regularizar los parámetros, ajuste de hiperparámetros, funciones CommitCount, estableciendo sesgo, optimizadores, funciones de pérdida y pesos para reducir la tasa de falsos positivos. El modelo ajustado finamente detecta la vulnerabilidad. La tasa de aprendizaje especificada establecida en el modelo propuesto es 0,0005 en 300 epochs, neuronas = 128, paro temprano = 30 y tamaño de lote = 64. El RFEMLP impone una técnica envolvente basada en aprendizaje automático llamada eliminación recursiva de características (RFE) en una red neuronal multicapa. El RFE continúa eliminando las características irrelevantes en cada iteración hasta que alcanza las características más impactantes. El RFE reduce las características redundantes para mejorar la eficiencia. Hemos implementado un clasificador de árbol de decisión para RFE. Basándose en la diferencia global entre el espacio de características, hemos establecido la clasificación de características desde las más importantes a las menos importantes.

No	Combinación
Selección 1	Adam + MSE
Selección 2	Adam + MAE
Selección 3	AdaDelta + MSE
Selección 4	AdaDelta + MAE
Selección 5	Momentum + MSE
Selección 6	Momentum + MAE
Selección 7	SDG + MSE
Selección 8	SDG + MAE

Tabla 2. Diferentes combinaciones de optimizadores y funciones de pérdida.

Entrada: Total de características seleccionadas por GCN

Proceso:

Función FeatureSelection(MLP_features):
  total_selected_features = 0
  Para cada stf en MLP_features: // stf es el total de características seleccionadas por MLP
    Si stf > 0:
      Para cada f1 en stf:
        TempM.append(f1.[a]) // Asignar a una ubicación de memoria temporal aleatoria
        Puntuación = eliminar(entrenar, validar, TempM) // Evaluar la puntuación de la característica para eliminación
        FX[f1[a]] = Puntuación // Almacenar rendimiento de la característica evaluada
        FMax = obtenerFMax(FX) // Identificar la característica con el máximo rendimiento
        a.eliminar(FMax) // Eliminar la característica con el máximo rendimiento de la consideración
        Si FX[FMax] > SetMax: // Comparar el rendimiento de la característica evaluada con el valor de rendimiento máximo, SetMax es el valor de rendimiento máximo
          SetMax = FX[FMax] // Actualizar el valor de rendimiento máximo
          A = f1 // Almacenar la característica seleccionada
          Incrementar contador
        Entonces
          Disminuir contador
      Fin de For
    Caso contrario:
      Devolver f1
  Fin de For
  Devolver total de características seleccionadas por RFEMLP

Algoritmo 1 Selección de características usando RFEMLP

Extracción de características basada en secuencias

CodeBERT

Los modelos preentrenados son eficaces en la predicción de vulnerabilidades108,109. CodeBERT combina la representación del codificador bidireccional de transformers y BERT optimizado llamado RoBERTa110. BERT es un modelo autossupervisado que utiliza las características de objetivos basados en máscaras y una arquitectura basada en transformadores. CodeBERT es el único modelo bimodal de gran tamaño preentrenado que usa lenguajes naturales y de programación111. Analiza de manera efectiva las conexiones semánticas entre el lenguaje de programación y elimina la dependencia de largo alcance en el código. Además, el mecanismo de atención multi-cabeza de los transformadores analiza de manera efectiva múltiples variables clave del flujo de datos.

La Fig. 6 ilustra la arquitectura del modelo CodeBERT. En el primer paso, CodeBERT toma la entrada de código y lo tokeniza. Hemos implementado el algoritmo de coincidencia más larga primero para tokenizar. En el segundo paso, los tokens se utilizan para extraer las características. Para realizar la extracción de características, hemos ajustado finamente

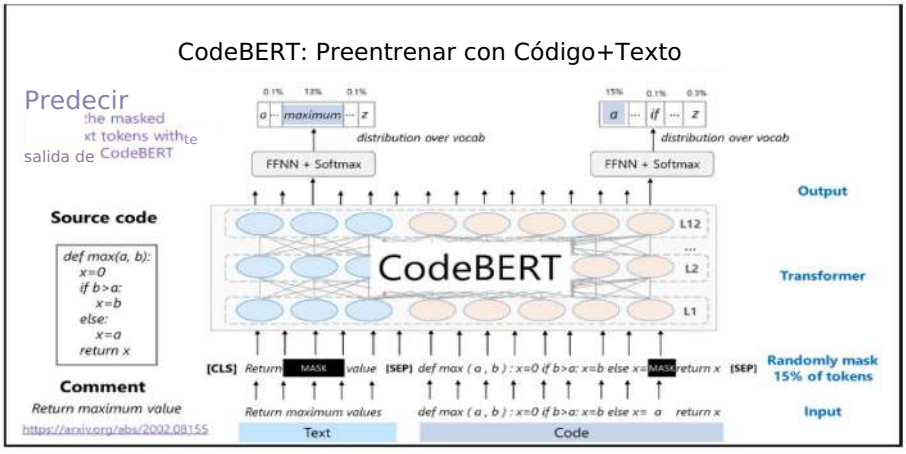

Figura 6. Estructura del modelo CodeBERT.

el CodeBERT configurando el tamaño de lote en 32, la tasa de aprendizaje en 10–3, 50 épocas y utilizando parada temprana para evitar el sobreajuste.

Clasificación

Red neuronal convolucional cuántica con agrupamiento auto-atentivo

El código fuente de Java tiene una estructura léxica compleja y características sintácticas y semánticas intrincadas con mayor longitud, lo que es difícil de abordar. Además, el software grande y complejo puede crear cuellos de botella computacionales y de memoria al tratar con la detección de vulnerabilidades. Por lo tanto, hemos empleado una red neuronal cuántica para superar estos problemas mediante mecanismos cuánticos. El mecanismo cuántico se basa en el entrelazamiento cuántico y los estados de superposición cuántica. Las redes neuronales cuánticas incorporan el entrelazamiento y los estados de superposición cuántica para mejorar la precisión de las redes neuronales. Utiliza el bit cuántico, la interferencia, la superposición y el mecanismo de entrelazamiento para el procesamiento de la información. El q-bit es un vector de estado representado en la ecuación siguiente

$$|\Psi\rangle = \theta|0\rangle + \delta|1\rangle \tag{2}$$

donde θ y δ son las amplitudes de probabilidad que están representadas por números complejos y |θ2| +|δ2|= 1. El mecanismo cuántico implica que cualquier matriz unitaria es una puerta cuántica U dada a continuación en la Ec. (3).

$$UU\dagger = U\dagger U = \quad I \tag{3}$$

donde U† es la traspuesta conjugada de una matriz U, e I es una matriz identidad. Hay tres tipos de puertas de qubit: 1. puerta de un qubit, que es la raíz cuadrada de la puerta NOT, también conocida como puertas de Pauli; 2. puerta de dos qubits que funciona en matrices unitarias de 4 × 4; y 3. puertas de múltiples qubits que funcionan en múltiples qubits como matrices unitarias de 2n × 2n. El mecanismo cuántico resolvió problemas de memoria en grandes cálculos y cuellos de botella estructurales, y alcanzó capacidades de cómputo superiores a la computación clásica.

La red neuronal convolucional cuántica proporciona un paradigma prometedor de aprendizaje automático. Hemos utilizado un dispositivo quantum pennyLane para simular el dispositivo de cuatro qubits. La puerta RY es responsable de convertir el código en bits cuánticos. La capa convolucional cuántica funciona como la capa convolucional convencional en el modelo CNN utilizando un mecanismo de computación cuántica. La convolución cuántica funciona como pequeños circuitos cuánticos aleatorios (RQC) para calcular la operación de convolución. Consta de tres fases: codificación, RQC y decodificación. El RQC se aplica a la capa convolucional y a la capa de agrupamiento. La capa de codificación es responsable de convertir las características extraídas en forma clásica en un estado de bit cuántico de alta dimensión. Hemos aplicado la codificación de base para convertir los datos en qubits. Las características concatenadas se convierten en características binarias y luego en un estado cuántico. El estado cuántico incrustado es la conversión bit a bit de la cadena binaria en un subsistema cuántico; así, el código fuente se transforma en el bit cuántico. La biblioteca paddle en Python se utiliza para la codificación de base.

En la segunda capa, el RQC se aplica en una capa convolucional que utiliza múltiples puertas de qubit entre los qubits adyacentes. De manera similar, las puertas de qubit aplicadas en el agrupamiento reducen el tamaño del sistema cuántico. Hemos aplicado un mecanismo de auto-atención en la capa de agrupamiento para mejorar el rendimiento del sistema. El circuito totalmente conectado es responsable de decodificar y clasificar el código vulnerable y el tipo de vulnerabilidad identificado. La QCNN utiliza el entrelazamiento multiescala MERA en dirección inversa y se repite hasta que se reduce suficientemente el tamaño del sistema cuántico.

Hemos aplicado una técnica novedosa de agrupamiento utilizando un mecanismo de auto-atención multi-cabeza para mejorar las huellas de cómputo y memoria, mejorando así el rendimiento del modelo. El mecanismo de auto-atención propuesto comprende tokenización, auto-atención multi-cabeza, restauración de canal espacial y funciones de activación sigmoide y softmax aplicadas en la capa de agrupamiento para hacerla auto-atentiva. Las características de entrada se tokenizan y la auto-atención multi-cabeza gestiona las dependencias a largo plazo en los tokens, mientras que la restauración del canal espacial ayuda a decodificar y restaurar los tokens en mapas de auto-atención. La función de activación softmax rectifica los mapas de auto-atención. Agregar un mecanismo de auto-atención en la QCNN mejora aún más las huellas de memoria y el cómputo. La red neuronal convolucional cuántica clasifica el código vulnerable e identifica el tipo de vulnerabilidad.

La Fig. 7 anterior ilustra la estructura general del modelo QCNN auto-atentivo propuesto para identificar la vulnerabilidad de seguridad y el tipo de vulnerabilidad.

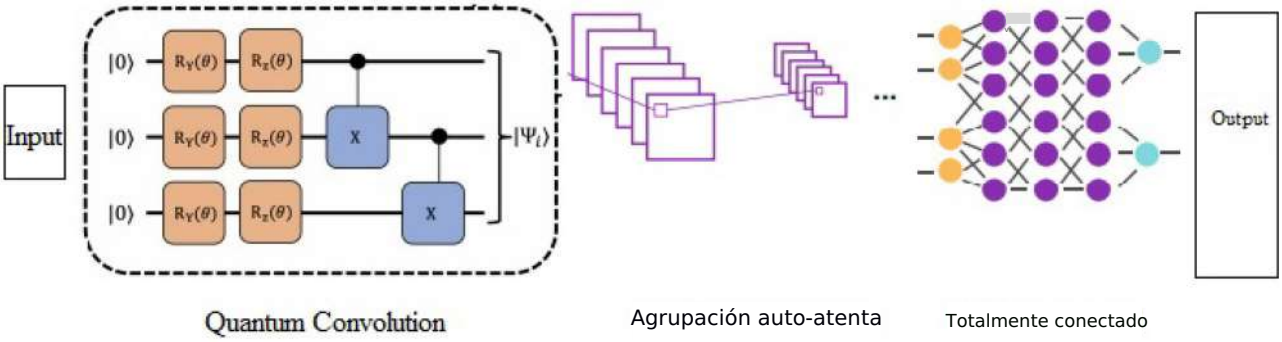

Figura 7. Composición estructural de una red neuronal cuántica con agrupación auto-atenta.

1. Conjunto_de_entrada = cargar_conjunto_de_datos() // Conjunto de entrada con código Java/JavaScript (p. ej., SARD)

2. Conjunto_balanceado = balancear_conjunto(de_datos_de_entrada) // Preprocesar los datos (equilibrio del conjunto, duplicados) eliminación, manejo de valores atípicos y valores faltantes)

conjunto_deduplicado = eliminar_duplicados(conjunto_balanceado)

conjunto_procesado = manejar_valores_atípicos_y_faltantes(conjunto_deduplicado)

3. Datos_vectorizados = vectorizar_datos(conjunto_procesado) // Vectorizar los datos convirtiendo datos de texto en enteros

4. Características_normalizadas = normalización_z_score(datos_vectorizados) // Normalizar la escala de características usando z-score

5. Representación_intermedia = generar_representación_inmediata(características_normalizadas) // Realizar extracción de características basada en grafos

incrustaciones_de_nodos = aplicar_incrustación_de_nodos(representación_intermedia)

características_extraídas = realizar_extracción_de_características(incrustaciones_de_nodos)

6. Características_de_secuencia = extraer_características_de_secuencia(características_normalizadas, modelo_CodeBERT) // Emplear extracción de características basada en secuencias usando un modelo preentrenado, CodeBERT

7. Características_concatenadas = concatenar(características_de_secuencia, características_extraídas) // Concatenar el características basadas en secuencias y basadas en grafos

8. Qubits_codificados = codificar_código_en_qubits(características_concatenadas) // Aplicar convolución cuántica red neuronal con agrupación auto-atenta

salida_rqc = aplicar_circuito_cuántico_aleatorio(qubits_codificados)

salida_auto_atenta = agrupación_auto_atenta(rqc_salida)

decoded_output = decode_output(self_attentive_output)

9. Código vulnerable, tipo de vulnerabilidad = identificar_vulnerabilidad(decoded_output)

produce_output(código_vulnerable, tipo_de_vulnerabilidad) //Producir salida como código vulnerable y tipo de vulnerabilidad

Algoritmo 2 Composición del sistema de detección de vulnerabilidades propuesto

Experimentos y resultados

Configuración experimental

El sistema automático propuesto de detección de vulnerabilidades fue evaluado mediante numerosos experimentos en una computadora con Windows equipada con un procesador Intel® Core™ i7-10700H y 128 GB de RAM. El modelo está implementado usando Python y el marco Tensorflow con paquetes de biblioteca como Keras, NumPy, sci-kit-learn y Pandas. Los hiperparámetros se establecen como epoch = 50, tasa de aprendizaje = 0.005, impulso = 0.9, tasa de abandono = 0.3, pérdida = entropía cruzada.

Métricas de rendimiento

Evaluamos el rendimiento del sistema propuesto utilizando varias métricas, incluida la recuperación, precisión y exactitud. La exactitud se calculó según la Ecuación (4).

$$\text{Precisión} = \frac{TP + TN}{TP + TN + FP + FN} \quad (4)$$

En esta ecuación, TN significa negativo verdadero, TP positivo verdadero, FP negativo falso y FN positivo falso. Métricas adicionales utilizadas para la validación del rendimiento fueron la precisión (véase la Ecuación 5), que representa la fracción de predicciones positivas correctas, y la recall (véase la Ecuación 6), que indica la proporción de predicciones positivas correctas entre todas las predicciones positivas.

$$\text{Precisión} = \frac{TP}{TP + FP} \quad (5)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (6)$$

Análisis comparativo

El sistema propuesto se desarrolló para predecir de manera efectiva la vulnerabilidad de seguridad de los sistemas de software. Para analizar el rendimiento del sistema propuesto, se sometió a pruebas en código fuente para identificar posibles vulnerabilidades de seguridad.

La Tabla 3 compara nuestra técnica con otras técnicas de aprendizaje profundo como CNN, SVM, GNN, LSTM, BiLSTM, ANN, MLP, DNN y FFDNN. El modelo propuesto mostró mayor precisión, exactitud y recall, lo que sugiere una mayor efectividad para detectar la máxima vulnerabilidad de seguridad.

La investigación se centró en diferentes tipos de vulnerabilidad, cada uno con características semánticas únicas. El sistema propuesto se entrenó con el conjunto de datos equilibrado SARD que contiene datos sintetizados, haciéndolo universalmente aplicable a varios tipos de vulnerabilidad. Para evaluar eficazmente la validez y el rendimiento de nuestro sistema, el sistema se entrenó usando otros conjuntos de datos, incluidas Juliet Java 1.3, FUNDED, Vul4J y CVEfixes. El SARD y Juliet Java 1.3 son conjuntos de datos de referencia puestos a disposición por NIST.

La Tabla 4 muestra que el sistema propuesto funcionó bien con otros conjuntos de datos FUNDED, Vul4J, CVE-fixes, CodeXGLUE, SARD, VUDDY y Julia Java 1.3, lo que demuestra la validez del sistema propuesto.

En la Tabla 5, nuestro modelo propuesto se compara con herramientas comerciales de detección de vulnerabilidades como VulDeepecker, SQVDT, Exp-Gen, PreNNsem, ISVSF, VULDEFF, SedSVD, VulANalyZeR, FUNDED, GraphSPD, BiTCN_DRSN y VERI. El sistema propuesto superó en precisión, exactitud y tasas de recall.

La Figura 8 muestra la precisión de entrenamiento y prueba del sistema propuesto. Los datos destacan el rendimiento superior de nuestro sistema, logrado al integrar extracción de características híbrida con información sintáctica y semántica de la

Clasificador	Precisión	F1
CNN112	0.92	0.92
SVM113	0.96	0.95
GNN114	0.95	–
LSTM115	0.96	0.96
BiLSTM116	0.96	–
ANN117	0.98	0.98
MLP118	0.84	–
DNN119	0.82	–
FFDNN120	0.77	–
modelo propuesto	0.99	0.97

Tabla 3. Análisis comparativo con técnicas de aprendizaje automático.

Conjunto de datos	Precisión	Recall	puntuación F1
FINANCIADO	0.95	0.92	0.91
Vul4J	0.96	0.96	0.95
CVEfixes	0.98	0.95	0.93
CodeXGLUE	0.96	0.98	0.99
SARD	0.98	0.97	0.95
VUDDY	0.92	0.97	0.97
Juliet java 1.3	0.98	0.95	0.96

Tabla 4. Evaluación del rendimiento del detector de vulnerabilidades propuesto utilizando conjuntos de datos bien conocidos.

	Precisión	Puntuación F1	Precisión	Recuperación
VulDeepecker121	0.95	0.93	0.92	0.94
SQVDT122	0.97	0.95	0.94	0.96
Exp-Gen123	–	–	0.95	–
PreNNsem124	0.96	0.97	0.96	0.98
ISVSF125	0.95	0.90	–	–
MFXSS126	0.98	0.98	0.97	0.96
VULDEFF127	–	0.88	0.91	0.85
SedSVD128	0.91	0.95	–	–
VulANalyZeR129	0.89	0.90	0.85	0.95
FUNDED130	0.92	–	–	0.94
GraphSPD131	0.80	–	–	–
BiTCN_DRSN132	0.95	0.95	0.92	0.98
VERI133	0.92	0.93	0.94	0.91
Modelo propuesto	0.99	0.97	0.98	0.96

Tabla 5. Análisis comparativo con detectores de vulnerabilidades existentes.

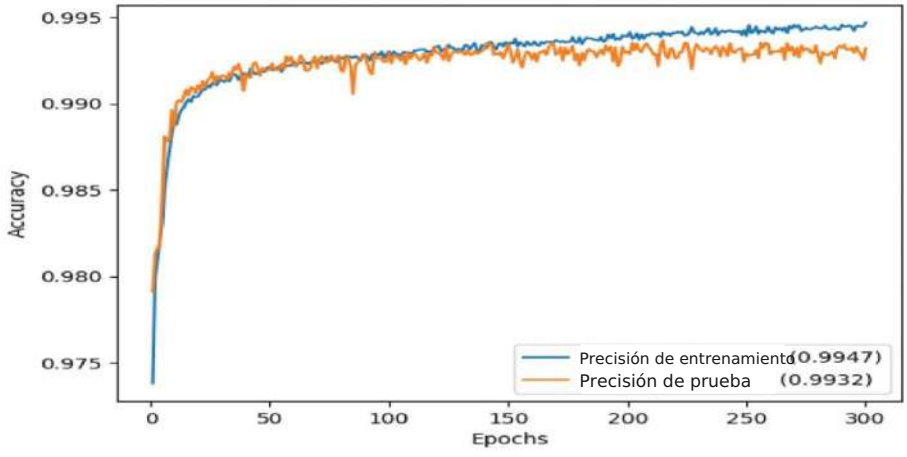

Figura 8. La precisión de entrenamiento y prueba del sistema propuesto.

código. Es notable que nuestro sistema redujo con éxito la tasa de falsos positivos mientras garantiza un número mínimo de valores faltantes.

Conclusión

Este estudio propone un sistema innovador diseñado para analizar vulnerabilidades en código de software, con el objetivo de abordar las limitaciones encontradas en técnicas anteriores de aprendizaje profundo. Los métodos de detección de vulnerabilidades no han considerado adecuadamente la semántica del código, lo que conduce a un rendimiento subóptimo. Nuestro sistema propuesto, que combina extracción de características basada en grafos y extracción basada en secuencias con una novedosa red neuronal GCN-RFEMLP, modelo preentrenado CodeBERT y agrupamiento automático con atención QCNN, audita con éxito el código fuente para posibles vulnerabilidades de seguridad. Aprovechamos la representación intermedia del código, utilizando un grafo de propiedades de código (CPG) para representación gráfica del código, que consta de un árbol de sintaxis abstracta (AST), un grafo de flujo de control (CFG) y un grafo de dependencias del programa.

El conjunto de datos se preprocesa considerando la importancia del equilibrio de datos, eliminación de código duplicado, valores faltantes, manejo de valores atípicos, vectorización y normalización para robustez, eficiencia y velocidad computacional. Además, se utiliza una red neuronal cuántica convolucional con agrupamiento autoatento como clase. Nuestra investigación se centra en tipos específicos de vulnerabilidad: validación de entrada inapropiada, cross-site scripting (XSS), autorización faltante, desbordamiento de entero y inyección SQL, que figuran entre las 25 vulnerabilidades de seguridad de software más significativas en la enumeración de debilidades comunes (CWE). El conjunto de datos de referencia de Software Assurance (SARD), un conjunto de referencia, se empleó para entrenar nuestro modelo. Además, para demostrar la validez del sistema, el sistema propuesto se utiliza con otros conjuntos de datos de referencia, incluidos FUNDED, Vul4j, CVEfixes, CodeXGLUE, SARD, VUDDY y Juliet Java 1.3.

Para validar la eficiencia de nuestro sistema, comparamos su rendimiento no solo con enfoques de aprendizaje profundo prevalentes como CNN, SVM, GNN, LSTM, BiLSTM, ANN, MLP, DNN y FFDNN, sino también con otros disponibles

sistemas como VulDeepecker, SQVDT, Exp-Gen, PreNNsem, ISVSF, VULDEFF, SedSVD, VulANalyZeR, FUNDED, GraphSPD, BiTCN_DRSN y VERI. Los resultados de nuestros experimentos demuestran el rendimiento superior de nuestro sistema propuesto en varias métricas, lo que significa un avance prometedor en el campo de la detección automática de vulnerabilidades.

Direcciones futuras

El sistema propuesto de detección de vulnerabilidades de seguridad, con su eficiente extracción de características y mecanismo cuántico, incluyendo el agrupamiento auto-atento, aborda con éxito los problemas existentes en la detección de vulnerabilidades en el código fuente Java. Si bien el sistema está adaptado a las complejidades estructurales del código fuente Java, extender el mecanismo propuesto a otros lenguajes de programación es una dirección futura crucial para evaluar su efectividad en diferentes bases de código. Además, explorar la aplicabilidad del sistema propuesto para resolver tareas de procesamiento de lenguaje natural (PLN) promete mitigar problemas de tiempo, costo y cuellos de botella de memoria en contextos más amplios.

Disponibilidad de datos

Los conjuntos de datos generados y/o analizados durante el presente estudio están disponibles en el repositorio de Github usando el enlace https:// github. com/ Vul- Detect- Code/ Vul- Detect.

Recibido: 19 de octubre de 2023; Aceptado: 12 de marzo de 2024
Published online: 28 March 2024

Referencias

1. Distribución de seguridad CVSS a lo largo del tiempo. (2023) https:// nvd. nist. gov/ gener al/ visua lizat ions/ vulne rabil ity- visua lizat ions/ cvss- sever ity- distr ibuti on- over- time.
2. Tassey, G. El impacto económico de la infraestructura inadecuada para las pruebas de software (RTI Health, Social, and Economics Research, 2002).
3. Zhivich, M. & Cunningham, R. K. El costo real de los errores de software. IEEE Secur. Priv. 7(2), 87–90 (2009).
4. Starsbug, J. & Bunge, J. Pérdidas inundan firma de comercio. Wall Street J 8(2). 1–15 (2012).
5. Geppert, L. La pérdida de contacto por radio deja a los pilotos por su cuenta. IEEE Spectrum 41(11), 16–17 (2004).
6. Berr, J. wannacry-ransomware-attacks-wannacry-virus-losses. https:// www. cbsne ws. com/ news/ wanna cry- ranso mware- attac ks- wanna cry- virus- losses/ (Consultado 2022).
7. Chen, Y., Chen, J., Gao, Y., Chen, D. & Tang, Y. Investigación sobre análisis de fallos de software y modelo de gestión de calidad. En IEEE International Conference on Software Quality, Reliability and Security Companion, Lisboa, Portugal (2018).
8. Marjanov, T., Pashchenko, I. & Massacci, F. Aprendizaje automático para la detección de vulnerabilidades en código fuente: Qué funciona y qué aún no existe. IEEE Secur. Priv. 20, 60–76 (2022).
9. Wang, X. et al. Aprendizaje profundo federado para la detección de anomalías en el internet de las cosas. Comput. Electr. Eng. 108, 108651 (2023).
10. Srivastava, A. & Bharti, M. R. Modelo híbrido de aprendizaje automático para la detección de anomalías en datos no etiquetados de redes de sensores inalámbricos. Wirel. Pers. Commun. 129, 2693–2710 (2023).
11. Gao, Y., Yin, X., He, Z. & Wang, X. Un enfoque de detección de anomalías en procesos mediante aprendizaje profundo con características latentes representativas para datos anómalos poco discriminativos e insuficientes. Comput. Ind. Eng. 176, 108936 (2023).
12. Tekerek, A. Una arquitectura novedosa para la detección de ataques web utilizando redes neuronales convolucionales. Comput. Secur. 100, 102096 (2021).
13. Gupta, R., Patel, M. M., Shukla, A. & Tanwar, S. Esquema de detección de contratos inteligentes maliciosos basado en aprendizaje profundo para entornos de internet de las cosas. Comput. Electr. Eng. 97, 107583 (2022).
14. Dairi, A., Harrou, F., Bouyeddou, B., Senouci, S.-M. & Sun, Y. Esquemas de detección de anomalías impulsados por aprendizaje profundo semi-supervisado para la detección de ciberataques en redes inteligentes. En Power System Cybersecurity, 265–295 (2023).
15. Lam, A. N., Nguyen, A. T., Nguyen, H. A. & Nguyen, T. N. Combinando aprendizaje profundo con recuperación de información para localizar archivos defectuosos a partir de informes de errores (N). En IEEE International Conference on Automated Software Engineering (ASE), Lincoln, NE, EE. UU. (2015).
16. Pu, Y., Narasimhan, K., Solar-Lezama, A. & Barzilay, R. sk_p: un corrector de programas neuronal para MOOCs. En SPLASH Companion 2016: Actas complementarias de la 2016 ACM SIGPLAN International Conference on Systems, Programming, Languages and Applications: Software for Humanity, Nueva York, (2016).
17. White, M., Vendome, C., Linares-Vásquez, M. & Poshyvanyk, D. Hacia repositorios de software de aprendizaje profundo. En MSR '15: Actas de la 12ª Conferencia de Trabajo sobre Minería de Repositorios de Software (2015).
18. Scandariato, R., Walden, J., Hovsepyan, A. & Joosen, W. Predicción de componentes de software vulnerables mediante minería de texto. IEEE Trans. Softw. Eng. 40(10), 993–1006 (2014).
19. Morrison, P., Herzig, K., Murphy, B. & Williams, L. Desafíos en la aplicación de modelos de predicción de vulnerabilidades. En HotSoS '15: Actas del Simposio y Bootcamp 2015 sobre la Ciencia de la Seguridad (2015).
20. Dam, H. K. et al. Aprendizaje automático de características para predecir componentes de software vulnerables. IEEE Trans. Softw. Eng. 47(1), 67–85 (2018).
21. Choi, M.-J., Jeong, S., Oh, H. & Choo, J. Predicción de desbordamientos de búfer de extremo a extremo a partir de código fuente sin procesar mediante redes neuronales de memoria. En IJCAI'17: Actas de la 26ª Conferencia Internacional Conjunta sobre Inteligencia Artificial, Melbourne, Australia (2017).
22. Pang, Y., Xue, X. & Namin, A. S. Predicción de componentes de software vulnerables mediante análisis de N-gramas y selección estadística de características. En 2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA) (2015).
23. Hovsepyan, A., Scandariato, R., Joosen, W. & Walden, J. Predicción de vulnerabilidades de software utilizando técnicas de análisis de texto. En MetriSec '12: Actas del 4º Taller Internacional sobre Medidas y Métricas de Seguridad, Nueva York (2012).
24. Piantadosi, V., Scalabrino, S. & Oli, R. Corrección de vulnerabilidades de seguridad en proyectos de código abierto: Un estudio de caso de Apache HTTP server y Apache tomcat. En International Conference on Software Testing, Verification, and Validation, ICST, (2019).
25. Hanif, H., Md Nasir, M. H. N., Ab Razak, M. F., Firdaus, A. & Anuar, N. B. El auge de la vulnerabilidad de software: Taxonomía de la detección de vulnerabilidades de software y enfoques de aprendizaje automático. J. Netw. Comput. Appl. 179, 103009 (2021).
26. Beaman, C., Redbourne, M., Mummery, J. D. & Hakak, S. Técnicas de descubrimiento de vulnerabilidades mediante fuzzing: Revisión, desafíos y direcciones futuras. Comput. Secur. 120, 102813 (2022).
27. Kronjee, J., Hommersom, A. & Vranken, H. Descubrimiento de vulnerabilidades de software utilizando análisis de flujo de datos y aprendizaje automático. En ARES '18: Actas de la 13ª Conferencia Internacional sobre Disponibilidad, Fiabilidad y Seguridad (2018).
28. Kim, S., Woo, S., Lee, H. & Oh, H. VUDDY: Un enfoque escalable para el descubrimiento de clones de código vulnerable. En IEEE Symposium on Security and Privacy (2017).

29. Shuai, B., Li, H., Zhang, L., Zhang, Q. & Tang, C. Detección de vulnerabilidades de software basada en la cobertura de código y el costo de pruebas. En la Conferencia Internacional sobre Inteligencia Computacional y Seguridad (2015).
30. Yu, Z., Theisen, C., Williams, L. & Menzies, T. Mejorando la eficiencia de inspección de vulnerabilidades mediante aprendizaje activo. IEEE Trans. Softw. Eng. 47, 2401–2420 (2015).
31. Liu, S. et al. DeepBalance: Aprendizaje profundo y sobremuestreo difuso para la detección de vulnerabilidades. IEEE Trans. Fuzzy Syst. 28(7), 1329–1343 (2019).
32. Yamaguchi, F., Golde, N., Arp, D. & Riek, K. Modelado y descubrimiento de vulnerabilidades con gráficos de propiedades de código. En IEEE Symposium on Security and Privacy (2014).
33. Hin, D., Kan, A., Chen, H. & Babar, M. A. LineVD: Detección de vulnerabilidades a nivel de sentencia usando redes neuronales gráficas. En MSR '22: Proceedings of the 19th International Conference on Mining Software Repositories, Nueva York (2022).
34. Fan, Y., Wan, C., Han, C. F. L. & Xu, H. VDoTR: Detección de vulnerabilidades basada en la representación tensorial de gráficos de código integrales. Comput. Secur. 130, 103247 (2023).
35. Suneja, S., Zheng, Y., Zhuang, Y., Laredo, J. & Morari, A. Aprendizaje para mapear código fuente a vulnerabilidades de software utilizando código- como-grafo. En International Conference on Science and Technology (2020).
36. Mikolov, T., Chen, K., Corrado, G. & Dea, J. Estimación eficiente de representaciones de palabras en un espacio vectorial. En ICLR Workshop Track 2013, Scottsdale, AZ, EE. UU. (2013).
37. Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M. & Monfardini, G. El modelo de red neuronal gráfica. IEEE Trans. Neural Netw. 20(1), 61–80 (2008).
38. Wang, J., Xiao, H., Zhong, S. & Xiao, Y. DeepVulSeeker: Un marco novedoso de identificación de vulnerabilidades mediante la estructura de gráficos de código y un mecanismo de pre-entrenamiento. Future Gener. Comput. Syst. 148, 15–26 (2023).
39. Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M. & Yin, J. UniXcoder: Preentrenamiento unificado multimodal para la representación de código. En Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (2022).
40. Liang, H., Wang, L. S. M. & Yang, Y. Aprendizaje profundo con árbol de sintaxis abstracta personalizado para la localización de errores. IEEE Access 7, 116309–116320 (2019).
41. Partenza, G., Amburgey, T., Deng, L., Dehlinger, J. & Chakraborty, S. Identificación automática de código vulnerable: investigaciones con una red neuronal basada en AST. En IEEE Annual International Computer Software and Applications Conference (COMPSAC) (2021).
42. Vishnu, P. R., Vinod, P. & Yerima, S. Y. Un enfoque de aprendizaje profundo para clasificar descripciones de vulnerabilidad usando red neuronal basada en autoatención. J. Netw. Syst. Manag. 30, 9 (2021).
43. Sobania, D., Briesch, M., Hanna, C. & Petke, J. Un análisis del rendimiento de corrección automática de errores de ChatGPT. En 2023 IEEE/ ACM International Workshop on Automated Program Repair (APR), 23–30 (2023).
44. Ren, J., Zheng, Z., Liu, Q., Wei, Z. & Yan, H. Enfoque de predicción de desbordamiento de búfer basado en métricas de software y aprendizaje automático. Secur. Commun. Netw. 2019, 1-13, 8391425 (2019).
45. Li, R., Feng, C., Zhang, X. & Tang, C. Un método ligero de descubrimiento de vulnerabilidades asistido usando redes neuronales profundas. IEEE Access 7, 80079–80092 (2019).
46. Li, X. et al. Detección automática de vulnerabilidades en código fuente usando aprendizaje de representación intermedia mínima. Appl. Sci. 10(5), 1692 (2020).
47. Tian, J., Xing, W. & Li, Z. BVDetector: Sistema inteligente de detección de vulnerabilidades en código binario basado en rebanadas de programa. Inf. Softw. Technol. 123, 106289 (2020).
48. Zagane, M., Abdi, M. K. & Alenezi, M. Aprendizaje profundo para la detección de vulnerabilidades de software usando métricas de código. IEEE Access 8, 74562–74570 (2020).
49. Bashir, O. A. Detección de ataques de cross-site scripting usando redes neuronales profundas. En 2023 3rd International Conference on Computing and Information Technology (ICCIT) (2023).
50. Zhou, Y., Liu, S., Siow, J., Du, X. & Liu, Y. Devign: Identificación efectiva de vulnerabilidades mediante el aprendizaje de semánticas de programa comprensivas a través de redes neuronales gráficas. NeurIPS 32, 1-11 (2019).
51. Guo, N., Li, X., Yin, H. & Gao, Y. VulHunter: Un sistema automático de detección de vulnerabilidades basado en aprendizaje profundo y bytecode. En International Conference of Information and Communication Security, China (2019).
52. Xuan, C. D., Mai, D. H., Thanh, M. C. & Cong, B. V. Un enfoque novedoso para la detección de vulnerabilidades de software basado en computación cognitiva inteligente. J. Supercomputing 79(15), 17042–17078 (2023).
53. Russell, R. et al. Detección automática de vulnerabilidades en código fuente usando aprendizaje de representaciones profundas. En IEEE International Conference on Machine Learning and Applications (IEEE ICMLA 2018), Orlando, Florida, USA (2018).
54. Hu, L., Chang, J., Chen, Z. & Hou, B. Método de detección de vulnerabilidades en aplicaciones web basado en aprendizaje automático. J. Phys. 1827(1), 012061 (2021).
55. Alves, H., Fonseca, B. & Antunes, N. Experimentando técnicas de aprendizaje automático para predecir vulnerabilidades. En Latin-American Symposium on Dependable Computing (LADC) (2016).
56. Saccente, N., Dehlinger, J., Deng, L., Chakraborty, S. & Xiong, Y. Project Achilles: una herramienta prototipo para detección estática de vulnerabilidades a nivel de método del código fuente Java usando una red neuronal recurrente. En 34th IEEE/ACM International Conference on Automated Software Engineering Workshop (ASEW) (2019).
57. Pang, Y., Xue, X. & Wang, H. Predicción de componentes de software vulnerables mediante redes neuronales profundas. En Proceedings of the 2017 International Conference on Deep Learning Technologies (2017).
58. Wartschinski, L., Noller, Y., Vogel, T., Kehrer, T. & Grunske, L. VUDENC: Detección de vulnerabilidades con aprendizaje profundo en una base de código natural para Python. Inf. Softw. Technol. 14, 106809 (2022).
59. Niu, W. et al. Enfoque de análisis de ataduras estáticas basado en aprendizaje profundo para la ubicación de vulnerabilidades en software IoT. Measurement 152, 107139 (2020).
60. An, J. H., Wang, Z. & Joe, I. Detección automática de vulnerabilidades basada en CNN. EURASIP J. Wireless Communications and Networking 2023(1), 41 (2023).
61. Phan, A. V., Nguyen, M. L. & Bui, L. T. Redes neuronales convolucionales sobre gráficos de flujo de control para la predicción de defectos de software. En International Conference on Tools for Artificial Intelligence (ICTAI), Boston, MA, USA (2017).
62. Hanif, H., Nasir, M. H. N. M., Razak, M. F. A., Firdaus, A. & Anuar, N. B. El auge de la vulnerabilidad de software: taxonomía de la detección de vulnerabilidades y enfoques de aprendizaje automático. J. Netw. Comput. Appl. 179, 103009 (2021).
63. Luo, Y., Xu, W. & Xu, D. Gráficos abstractos compactos para detectar vulnerabilidades de código con modelos GNN. En ACSAC'22: Proceedings of the 38th Annual Computer Security Applications Conference ACM, Texas (2022).
64. Nguyen, V. A., Nguyen, D. Q., Nguyen, V., Le, T., Tran, Q. H. & Phung, D. ReGVD: Revisitando redes neuronales gráficas para la detección de vulnerabilidades. En ICSE'22: Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, Nueva York (2021).
65. Boghdady, A. A., Ramly, M. E. & Wassif, K. iDetect para detección de vulnerabilidades en sistemas operativos de Internet de las cosas usando aprendizaje automático. Sci. Rep. 12(1), 17086 (2022).
66. Perl, H. et al. VCCFinder: Encontrar vulnerabilidades potenciales en proyectos de código abierto para ayudar auditorías de código. En ACM (2015).
67. Grieco, G., Grinblat, G. L., Uzal, L. C., Rawat, S., Feist, J. & Mounier, L. Hacia el descubrimiento de vulnerabilidades a gran escala usando aprendizaje automático. En CODASPY '16: Proceedings of the Sixth ACM Conference on Data and Application Security and Privacy (2016).

68. Chernis, B. & Verma, R. M. Métodos de aprendizaje automático para la detección de vulnerabilidades de software. En IWSPA '18: Actas del Cuarto Taller Internacional ACM sobre Seguridad y Análisis de Privacidad, Nueva York (2018).
69. Iorga, D., Corlătescu, D., Grigorescu, O., Săndescu, C., Dascălu, M. & Rughiniş, R. Detección temprana de vulnerabilidades en sitios web de noticias utilizando modelos de aprendizaje automático. En la Conferencia Internacional Roedunet (RoEduNet) (2020).
70. CWE. https:// cwe. mitre. org (2022) https:// cwe. mitre. org/ top25/ archi ve/ 2021/ 2021_ cwe_ top25. html.
71. Yan, H. et al. Detección de ataques de cross-site scripting basada en una red neuronal convolucional modificada. Front. Comput. Neuro Sci. 16, 981739 (2022).
72. Shahriar, H. & Zulkernine, M. S2XS2: Un enfoque del lado del servidor para detectar automáticamente ataques XSS. En el Simposio Internacional IEEE sobre Computación Confiable, Autónoma y Segura (DASC), Australia (2011).
73. Wang, R., Xu, G., Zeng, X., Li, X. & Feng, Z. TT-XSS: Un novedoso marco de detección dinámica basado en seguimiento de contaminación para Cross-Site Scripting en el DOM. J. Parallel Distrib. Comput. 118, 100–106 (2018).
74. Kaur, G., Malik, Y., Samuel, H. & Jaafar, F. Detección de ataques de cross-site scripting ciegos utilizando aprendizaje automático. En SPML '18: Actas de la Conferencia Internacional 2018 sobre Procesamiento de Señales y Aprendizaje Automático, Shanghái, China (2018).
75. Fang, Y., Li, Y., Liu, L. & Huang, C. DeepXSS: Detección de cross-site scripting basada en aprendizaje profundo. En ICCAI '18: Actas de la Conferencia Internacional 2018 sobre Computación e Inteligencia Artificial, Chengdu, China (2018).
76. Tariq, I. et al. Resolución de ataques de cross-site scripting mediante algoritmos genéticos y aprendizaje por refuerzo. Expert Syst. Appl. 168, 114386 (2015).
77. Prasetio, D. A., Kusrini, K. & Arief, M. R. Detección de ataques de cross-site scripting utilizando aprendizaje automático con características híbridas. J. Infotel 13, 1–6 (2021).
78. Falana, O. J., Ebo, I. O., Tinubu, C. O., Adejimi, O. A. & Ntuk, A. Detección de ataques de cross-site scripting utilizando análisis dinámico y sistema de inferencia difusa. En la Conferencia Internacional de Matemáticas, Ingeniería Informática y Ciencias de la Computación (ICM-CECS), Ayobo, Nigeria (2020).
79. Tsai, D. R., Chang, A. Y., Liu, P. & Chen, H. C. Ajuste óptimo de configuraciones de defensa para ataques comunes en aplicaciones web. En la 43ª Conferencia Internacional Carnahan sobre Tecnología de Seguridad (2009).
80. Day, D. J., Zhao, Z. & Ma, M. Detección de ataques de desbordamiento de búfer return-to-libc utilizando sistemas de detección de intrusiones en red. En la Cuarta Conferencia Internacional sobre Sociedad Digital, ICDS, Antillas Neerlandesas (2010).
81. Sun, H., Zhang, X., Su, C. & Zeng, Q. Técnica eficiente de seguimiento dinámico para detectar vulnerabilidades de desbordamiento de enteros a desbordamiento de búfer. En ASIA CCS '15: Actas del 10º Simposio ACM sobre Seguridad de la Información, Computadoras y Comunicaciones, Singapur, República de Singapur (2015).
82. Ye, T., Zhang, L., Wang, L. & Li, X. Un estudio empírico sobre la detección y corrección de errores de desbordamiento de búfer. En la Conferencia Internacional IEEE sobre Pruebas, Verificación y Validación de Software (ICST) (2016).
83. Gao, F., Wang, L. & Li, X. BovInspector: Inspección y reparación automática de vulnerabilidades de desbordamiento de búfer. En la 31ª Conferencia Internacional IEEE/ACM (2016).
84. Kasím, Ö. Un enfoque basado en clasificación por conjuntos para detectar el nivel de ataque de inyecciones SQL. J. Inf. Security Appl. 59, 102852 (2021).
85. Xiao, Z., Zhou, Z., Yang, W. & Deng, C. Un enfoque para la detección de inyecciones SQL basado en el análisis de comportamiento y respuesta. En la Conferencia Internacional sobre Software de Comunicación y Redes, ICCSN, Guangzhou, China (2017).
86. Ciampa, A., Visaggio, C. A. & Penta, M. D. Un enfoque basado en heurísticas para detectar vulnerabilidades de inyección SQL en aplicaciones web. En SESS '10: Actas del Taller ICSE 2010 sobre Ingeniería de Software para Sistemas Seguros, Ciudad del Cabo, Sudáfrica (2010).
87. Nofal, D. E. & Amer, A. A. Detección y prevención de ataques de inyección SQL basada en técnicas neuro-difusas. Mach. Learn. Big Data Anal. Paradigms Anal. Appl. Challenges 77, 93-112 (2021).
88. Qureshi, K. N., Ghani, I. & Aliero, M. S. Un algoritmo para detectar vulnerabilidades de inyección SQL utilizando pruebas de caja negra. J. Ambient Intell. Human. Comput. 11, 249–266 (2019).
89. Gu, H. et al. DIAVA: Un marco basado en tráfico para la detección de ataques de inyección SQL y análisis de vulnerabilidades de datos filtrados. IEEE Trans. Reliab. 69(1), 188–202 (2019).
90. Ghafarian, A. Un método híbrido para la detección y prevención de ataques de inyección SQL. En la Conferencia de Ciencia e Información (SAI), Londres, Reino Unido (2017).
91. Ping, C. Un método de detección de inyección SQL de segundo orden. En la Conferencia IEEE de Tecnología de la Información, Redes, Electrónica y Control de Automatización, Chengdu, China (2017).
92. Dysart, F. & Sherriff, M. Generador automático de soluciones para ataques de inyección SQL. En el Simposio Internacional sobre Ingeniería de Fiabilidad de Software (ISSRE) (2008).
93. Cui, S., Zhao, G., Gao, Y., Tavu, T. & Huang, J. VRust: Detección automática de vulnerabilidades para contratos inteligentes de Solana. En CCS '22: Actas de la Conferencia ACM SIGSAC 2022 sobre Seguridad Informática y de las Comunicaciones, Nueva York (2022).
94. Lu, J., Li, H., Liu, C., Li, L. & Cheng, K. Detección de vulnerabilidades por falta de comprobación de permisos en sistemas distribuidos en la nube. En CCS '22: Actas de la Conferencia ACM SIGSAC 2022 sobre Seguridad Informática y de las Comunicaciones, Nueva York (2022).
95. Lu, K., Pakki, A. & Wu, Q. Detección de errores por comprobaciones faltantes mediante inferencias semánticas y contextuales de criticidad y restricciones. En el Simposio de Seguridad USENIX (2019).
96. Monshizadeh, M., Naldurg, P. & Venkatakrishnan, V. N. MACE: Detección de vulnerabilidades de escalada de privilegios en aplicaciones web. En CCS '14: Actas de la Conferencia ACM SIGSAC 2014 sobre Seguridad Informática y de las Comunicaciones, Scottsdale, Arizona, EE.UU. (2014).
97. Situ, L., Wang, L., Liu, Y., Mao, B. & Li, X. Vanguard: detección de comprobaciones faltantes para pronosticar vulnerabilidades potenciales. En Internetware '18: Actas del 10º Simposio Asia-Pacífico sobre Internetware, Pekín, China (2018).
98. Son, S., McKinley, K. S. & Shmatikov, V. RoleCast: Encontrar comprobaciones de seguridad faltantes cuando no se sabe cuáles son. En OOPSLA'11: Actas de la Conferencia Internacional ACM 2011 sobre Sistemas, Lenguajes y Aplicaciones de Programación Orientada a Objetos (2011).
99. Park, J., Shin, J. & Choi, B. Detección de vulnerabilidades por uso incorrecto de variables utilizando aprendizaje automático. MDPI 12(5), 1197 (2023).
100. Al-Boghdady, A., El-Ramly, M. & Wassif, K. iDetect para la detección de vulnerabilidades en sistemas operativos de internet de las cosas utilizando aprendizaje automático. Sci. Rep. 12(1), 17086 (2022).
101. Ziems, N. y Wu, S. Detección de vulnerabilidades de seguridad con procesamiento de lenguaje natural mediante aprendizaje profundo. En IEEE INFOCOM 2021-Conferencias IEEE sobre Comunicaciones de Computadoras (INFOCOM WKSHPS) (2021).
102. Li, X. et al. Detección automática de vulnerabilidades de software basada en red neuronal híbrida. Appl. Sci. 11(7), 3201 (2021).
103. Jeon, S. & Kim, H. K. AutoVAS: Un sistema de análisis de vulnerabilidades automatizado con un enfoque de aprendizaje profundo. Comput. Secur. 106, 102308 (2021).
104. Li, Z.et alSySeVR: un marco para usar aprendizaje profundo para detectar vulnerabilidades de softwareIEEE Trans. Dependable Secure Comput. Secur. 19, 2244–2258 (2022).
105. Haojie, Z., Yujun, L., Yiwei, L. y Nanxin, Z. Vulmg: Una solución de detección estática para vulnerabilidades de código fuente basada en código grafo de propiedades y red de atención de grafos. En la Conferencia Internacional sobre Tecnología y Procesamiento de Medios Activos Wavelet (ICWAMTIP) (2021).

106. Grahn, D. & Zhang, J. Un análisis de conjuntos de datos C/C++ para software asistido por aprendizaje automático. En las Actas de la Conferencia sobre Aprendizaje Automático Aplicado para la Seguridad de la Información, 2021 (2021).
107. Amankwah, R., Chen, J., Song, H. & Kudjo, P. K. Detección de errores en código Java: una evaluación extensa de herramientas de análisis estático utilizando Juliet Test Suites. Softw. Pract. Exp. 53(5), 1125-1143 (2022).
108. Ray, B., Hellendoorn, V., Godhane, S., Tu, Z., Bacchelli, A. & Devanbu, P. Sobre la "naturalidad" del código con errores. En ICSE '16: Actas de la 38ª Conferencia Internacional sobre Ingeniería de Software, mayo de 2016 (2016).
109. Allamanis, M., Devanbu, E. T. B. P. & Sutton, C. Encuesta sobre aprendizaje automático para gran código y naturalidad. ACM Comput. Surv. 51(4), 1–37 (2018).
110. Feng, Z. et al. CodeBERT: Un modelo preentrenado para programación y lenguajes naturales. En Association for Computational Linguistics EMNLP, 1536–1547 (2020).
111. Liu, Y. et al. RoBERTa: Un enfoque de preentrenamiento BERT optimizado de forma robusta. En International Conference on Learning Representaciones, Adis Abeba (2019).
112. Yang, K., Miller, P. & Martinez-Del-Rincon, J. Red neuronal convolucional para la detección de vulnerabilidades de software. En Cyber Research Conference-Ireland (Cyber-RCI) (2022).
113. Rawat, R. & Shrivastav, S. K. Detección de ataques de inyección SQL usando SVM. Int. J. Comput. Appl. 42(13), 1–4 (2012).
114. Luo, Y., Xu, W. & Xu, D. Grafos abstractos compactos para detectar vulnerabilidades de código con modelos GNN. En ACSAC '22: Actas de la 38ª Conferencia Anual de Aplicaciones de Seguridad Informática, Nueva York (2022).
115. Xu, A., Dai, T., Chen, H., Ming, Z. & Li, W. Detección de vulnerabilidades en código fuente utilizando LSTM contextual. En 5th International Conference on Systems and Informatics (ICSAI) (2018).
116. Wang, M., Xie, Z., Wen, X., Li, J. & Zhou, K. Modelo de detección de vulnerabilidades de contratos inteligentes de Ethereum basado en pérdida de tripletes y BiLSTM. Electronics 12(10), 2327 (2023).
117. Kasongo, S. M. & Sun, Y. Análisis de rendimiento de sistemas de detección de intrusiones usando un método de selección de características en el UNSW-NB15 dataset. J. Big Data 7(105), 1-20 (2020).
118. Nawir, M., Amir, A., Yaakob, N. & Lynn, O. B. Clasificación múltiple del conjunto UNSW-NB15 para la detección de anomalías de red sistema. J. Theor. Appl. Inf. Technol. 96(15), 5094–5104 (2018).
119. Kasongo, S. M. & Sun, Y. Un método de aprendizaje profundo con extracción de características basada en envoltorio para sistemas de detección de intrusiones inalámbricas. Comput. Secur. 92, 101752 (2020).
120. Eunice, A. D., Gao, Q., Zhu, M.-Y., Chen, Z. & Lv, N. Tecnología de detección de anomalías de red basada en aprendizaje profundo. En 2021 IEEE Tercera Conferencia Internacional sobre Fronteras de Tecnología de la Información y la Computación (ICFTIC) (2021).
121. Li, Z. et al. VulDeePecker: un sistema basado en aprendizaje profundo para la detección de vulnerabilidades. En Network and Distributed System Security (NDSS) Symposium (2018).
122. Akram, J. & Luo, P. SQVDT: Una técnica de detección de vulnerabilidades cuantitativa y escalable para la evaluación de seguridad del código fuente. Softw. Práct. Exp. 51(2), 294-318 (2020).
123. Huang, H. et al. ExpGen: Una solución de evaluación de explotabilidad de vulnerabilidades en 2 pasos para programas binarios en un entorno ASLR. Appl. Sci. 12(13), 6593 (2022).
124. Wang, L. et al. PreNNsem: Un marco de aprendizaje en conjunto heterogéneo para la detección de vulnerabilidades en software. Appl. Sci. 10(22), 7954 (2023).
125. Zhang, H., Bi, Y., Guo, H., Sun, W. & Li, J. ISVSF: Detección inteligente de vulnerabilidades contra Java vía patrón a nivel de oración exploración. IEEE Syst. J. 16(1), 1032–1043 (2021).
126. Liu, Z., Fang, Y., Huang, C. & Xu, Y. MFXSS: Un método eficaz de detección de vulnerabilidades XSS en JavaScript basado en múltiples características modelo. Comput. Secur. 124, 103015 (2023).
127. Zhao, Q., Huang, C. & Dai, L. VULDEFF: Método de detección de vulnerabilidades basado en huellas de funciones y diferencias de código. Knowl.-Based Syst. 260, 110139 (2022).
128. Dong, Y., Tang, Y., Cheng, X., Yang, Y. & Wang, S. SedSVD: Detección de vulnerabilidades de software a nivel de sentencia basada en Relational Graph Convolutional Network with subgraph embedding. Inf. Softw. Technol. 158, 107168 (2023).
129. Li, L. et al. VulANalyzeR: Detección explicable de vulnerabilidades binarias con aprendizaje multitarea y convolución gráfica atencional. ACM Trans. Privacy Secur. 26, 1–25 (2023).
130. Wang, H. et al. Combinando aprendizaje basado en grafos con recopilación automática de datos para detección de vulnerabilidades de código. IEEE Trans. Inf. Forensics Secur. 16, 1943–1958 (2020).
131. Wang, S., Wang, X., Sun, K., Jajodia, S., Wang, H. & Li, Q. GraphSPD: Detección de parches de seguridad basada en grafos con código enriquecido semantics. En el IEEE Symposium on Security and Privacy (2023).
132. Chen, J. et al. BiTCN_DRSN: Un modelo eficaz de detección de vulnerabilidades de software basado en una convolución temporal mejorada network. J. Syst. Softw. 204, 111772 (2023).
133. Cheng, Y., Yang, S., Lang, Z., Shi, Z. & Sun, L. VERI: Detección de vulnerabilidades de componentes de código abierto a gran escala en IoT firmware. Comput. Secur. 126, 103068 (2023).

Contribuciones de los autores

Shumaila Hussain: concibió y diseñó el análisis; realizó el análisis estadístico formal, escribió el artículo, borrador original; revisión y edición de escritura. Muhammad Nadeem: realizó el análisis estadístico formal; contribuyó con reactivos, materiales, herramientas de análisis o datos; escribió el artículo; revisión y edición. Junaid Baber, Mohammed Hamdi: concibieron y diseñaron el análisis; realizaron el análisis; analizaron e interpretaron los datos; contribuyeron con reactivos, materiales, herramientas de análisis; escribieron el artículo; revisión y edición.

Financiación

Los autores agradecen a la Deanship of Scientific Research de Najran University por financiar este trabajo bajo el código de beca del programa de financiación de grupos de investigación (NU/RG/SERC/12/34).

Intereses competidores

Los autores declaran no tener intereses competidores.

Información adicional

Correspondencia y solicitudes de materiales deben dirigirse a S.H.

La información sobre reimpresiones y permisos está disponible en www.nature.com/reprints.

Nota del editor: Springer Nature continúa siendo neutral respecto a las reclamaciones de jurisdicción en mapas publicados y **afiliaciones institucionales.**

Acceso abierto Este artículo está licenciado bajo una Licencia Creative Commons Atribución 4.0 Internacional Licencia, que permite el uso, compartición, adaptación, distribución y reproducción en cualquier medio o format, siempre que des el crédito adecuado al autor(es) original(es) y a la fuente, proporciones un enlace a la licencia de Creative Commons y indiques si se realizaron cambios. Las imágenes u otro material de terceros en este artículo están incluidos en la licencia Creative Commons del artículo, a menos que se indique lo contrario en una línea de crédito al material. Si el material no está incluido en la licencia Creative Commons del artículo y tu uso previsto no está permitido por la regulación estatutaria o excede el uso permitido, deberás obtener permiso directamente del titular de los derechos de autor. Para ver una copia de esta licencia, visita http:// creat iveco mmons. org/ licen ses/ by/4. 0/.

© Los/Las Autor(es) 2024